

eda_notebook

June 21, 2026

1 Exploratory Data Analysis

1.0.1 Author: Ariel Cintrón-Arias

1.0.2 Required Modules

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
```

1.0.3 Data Dictionary

dataset:

name: fetch_diabetes_hospital

source: 130-US Hospitals Diabetes Dataset (1999-2008)

dimensions:

total_rows: 101766

total_columns: 25

features_count: 24

target_count: 1

columns:

- name: race
 - type: Categorical
 - role: Feature (Sensitive)
 - description: Race of the patient
 - valid_values: [African American, Asian, Caucasian, Hispanic, Other, Unknown]
- name: gender
 - type: Categorical
 - role: Feature (Sensitive)
 - description: Gender of the patient
 - valid_values: [Female, Male]
- name: age
 - type: Categorical
 - role: Feature (Sensitive)
 - description: Grouped age brackets
 - valid_values: ['30 years or younger', '30-60 years', 'Over 60 years']
- name: admission_source_id
 - type: Categorical
 - role: Feature
 - description: Entry point to the hospital

```

    valid_values: [Emergency, Referral, Other]
- name: discharge_disposition_id
  type: Categorical
  role: Feature
  description: Release destination after hospital stay
  valid_values: [Discharged to Home, Other]
- name: medical_specialty
  type: Categorical
  role: Feature
  description: Specialty of the admitting physician
  valid_values: [Cardiology, Internal Medicine, Emergency, Family/General Practice, Missing]
- name: time_in_hospital
  type: Integer
  role: Feature
  description: Number of days spent in hospital
  valid_values: 1-14
- name: num_lab_procedures
  type: Integer
  role: Feature
  description: Number of lab tests performed during visit
- name: num_procedures
  type: Integer
  role: Feature
  description: Number of non-lab procedures performed during visit
- name: num_medications
  type: Integer
  role: Feature
  description: Number of unique generic medications administered
- name: primary_diagnosis
  type: Categorical
  role: Feature
  description: Grouped primary diagnosis code
  valid_values: [Diabetes, Genitourinary, Musculoskeletal, Respiratory, Other]
- name: number_diagnoses
  type: Integer
  role: Feature
  description: Total number of diagnoses entered into the system
- name: max_glu_serum
  type: Categorical
  role: Feature
  description: Glucose serum test results
  valid_values: [None, Norm, ">200", ">300"]
- name: A1Cresult
  type: Categorical
  role: Feature
  description: Hemoglobin A1c test results
  valid_values: [None, Norm, ">7", ">8"]
- name: insulin

```

```

type: Categorical
role: Feature
description: Status of insulin prescription
valid_values: [No, Down, Steady, Up]
- name: change
  type: Binary
  role: Feature
  description: Indicates if there was a change in diabetic medications
  valid_values: [No, Ch]
- name: diabetesMed
  type: Binary
  role: Feature
  description: Was any diabetic medication prescribed?
  valid_values: [True, False]
- name: medicare
  type: Binary
  role: Feature
  description: Patient has Medicare coverage
  valid_values: [True, False]
- name: medicaid
  type: Binary
  role: Feature
  description: Patient has Medicaid coverage
  valid_values: [True, False]
- name: had_emergency
  type: Binary
  role: Feature
  description: Patient had emergency room visits in the prior year
  valid_values: [True, False]
- name: had_inpatient_days
  type: Binary
  role: Feature
  description: Patient had hospital inpatient days in the prior year
  valid_values: [True, False]
- name: had_outpatient_days
  type: Binary
  role: Feature
  description: Patient had outpatient clinic visits in the prior year
  valid_values: [True, False]
- name: readmit_30_days
  type: Binary
  role: Target Label
  description: Was the patient re-admitted to the hospital within 30 days?
  valid_values: [True, False]

```

In the Diabetes dataset from the Python module Fairlearn, the following columns are commonly considered sensitive features: * **race**: Categorized into African American, Caucasian, Hispanic, Asian, Other, and Unknown.

- **gender**: Categorized into Female, Male, and Unknown/Invalid.
- **age**: Grouped into ranges (30 years or younger, 30–60 years, and Over 60 years).

Furthermore, the following features may also risk inducing disparities based on socioeconomic factors, by means of service quality or allocation harms: * **medicare** and **medicaid**: These binary attributes indicate the patient’s type of health insurance. While technically socioeconomic features rather than fixed demographics, checking for bias against individuals on public assistance is a common domain-specific fairness audit in healthcare systems.

```
[2]: df = pd.read_csv('../data/diabetes_raw.csv')
```

```
[3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101766 entries, 0 to 101765
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   race                                  101766 non-null  object
1   gender                                101766 non-null  object
2   age                                   101766 non-null  object
3   discharge_disposition_id             101766 non-null  object
4   admission_source_id                  101766 non-null  object
5   time_in_hospital                     101766 non-null  int64
6   medical_specialty                    101766 non-null  object
7   num_lab_procedures                   101766 non-null  int64
8   num_procedures                       101766 non-null  int64
9   num_medications                      101766 non-null  int64
10  primary_diagnosis                     101766 non-null  object
11  number_diagnoses                      101766 non-null  int64
12  max_glu_serum                         5346 non-null    object
13  A1Cresult                             17018 non-null   object
14  insulin                               101766 non-null  object
15  change                                101766 non-null  object
16  diabetesMed                           101766 non-null  object
17  medicare                              101766 non-null  bool
18  medicaid                             101766 non-null  bool
19  had_emergency                         101766 non-null  bool
20  had_inpatient_days                    101766 non-null  bool
21  had_outpatient_days                   101766 non-null  bool
22  readmitted                           101766 non-null  object
23  readmit_binary                        101766 non-null  int64
24  readmit_30_days                       101766 non-null  int64
dtypes: bool(5), int64(7), object(13)
memory usage: 16.0+ MB
```

```
[4]: df.head()
```

```
[4]:
```

	race	gender	age	discharge_disposition_id	\
0	Caucasian	Female	'30 years or younger'	Other	
1	Caucasian	Female	'30 years or younger'	'Discharged to Home'	
2	AfricanAmerican	Female	'30 years or younger'	'Discharged to Home'	
3	Caucasian	Male	'30-60 years'	'Discharged to Home'	
4	Caucasian	Male	'30-60 years'	'Discharged to Home'	

	admission_source_id	time_in_hospital	medical_specialty	num_lab_procedures	\
0	Referral	1	Other	41	
1	Emergency	3	Missing	59	
2	Emergency	2	Missing	11	
3	Emergency	2	Missing	44	
4	Emergency	1	Missing	51	

	num_procedures	num_medications	... change	diabetesMed	medicare	medicaid	\
0	0	1	... No	No	False	False	
1	0	18	... Ch	Yes	False	False	
2	5	13	... No	Yes	False	False	
3	1	16	... Ch	Yes	False	False	
4	0	8	... Ch	Yes	False	False	

	had_emergency	had_inpatient_days	had_outpatient_days	readmitted	\
0	False	False	False	NO	
1	False	False	False	>30	
2	False	True	True	NO	
3	False	False	False	NO	
4	False	False	False	NO	

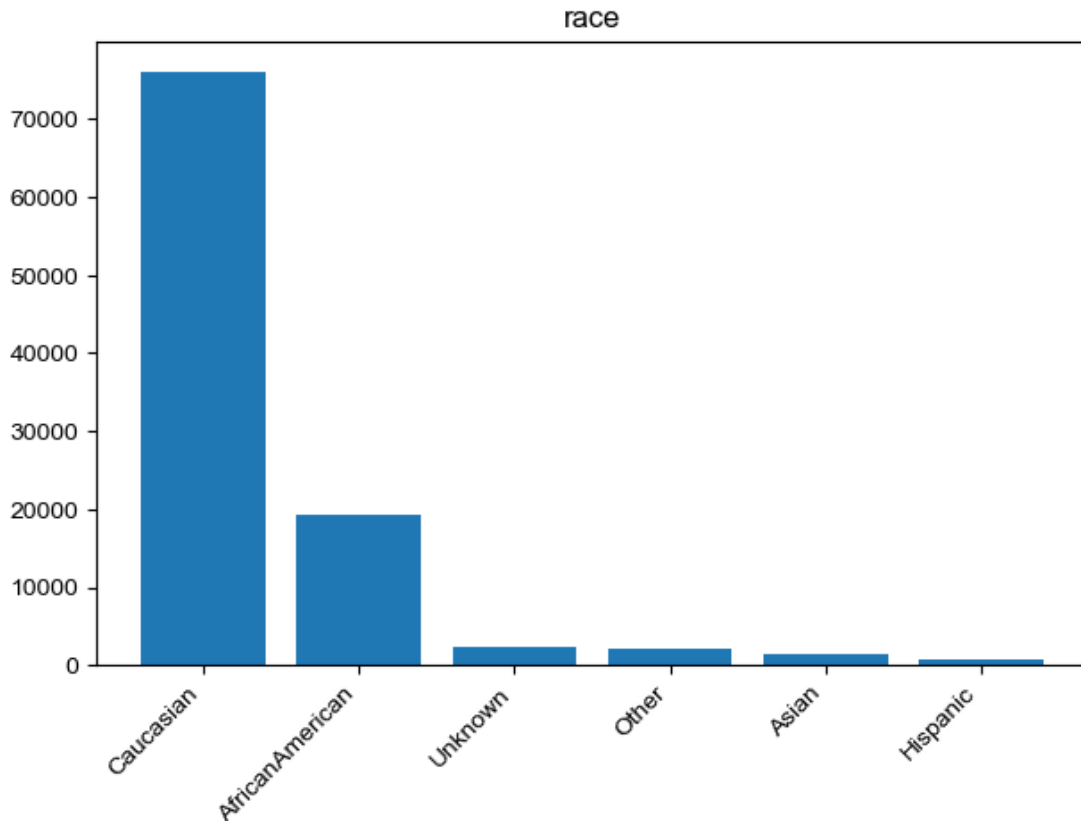
	readmit_binary	readmit_30_days
0	0	0
1	1	0
2	0	0
3	0	0
4	0	0

[5 rows x 25 columns]

1.0.4 Categorical Statistical Variables

```
[5]: #df.iloc[95:150,13]
#df['A1Cresult'].dropna().unique()
#df['age'].unique()
#df['gender'].unique()
#df[['num_lab_procedures','num_procedures','num_medications']].describe()
```

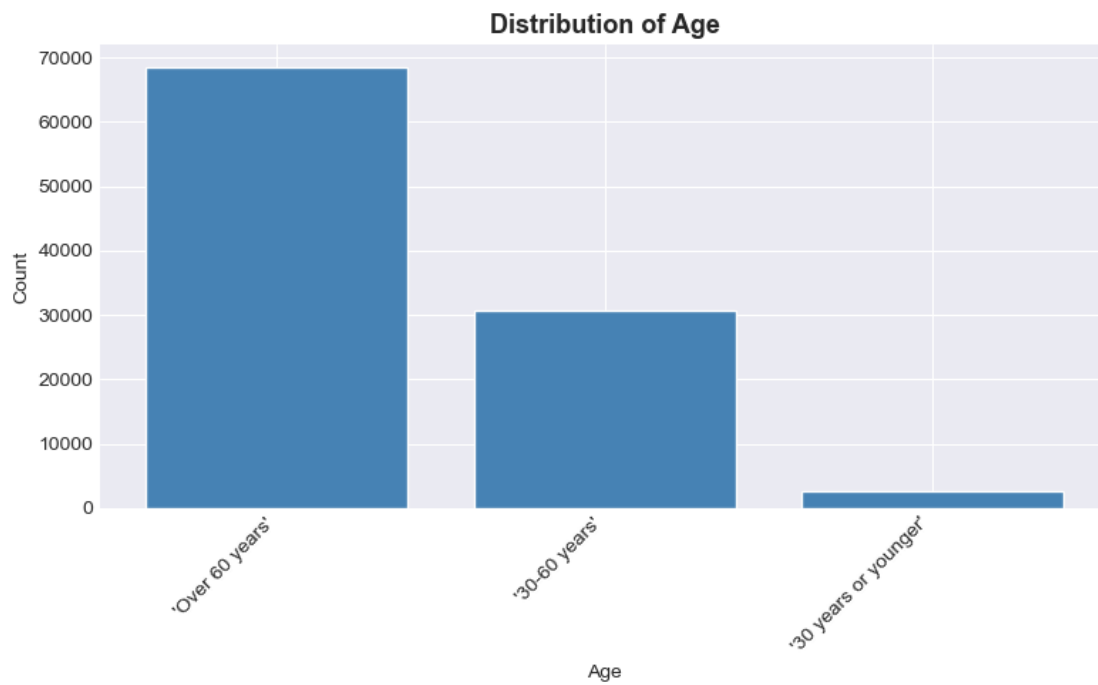
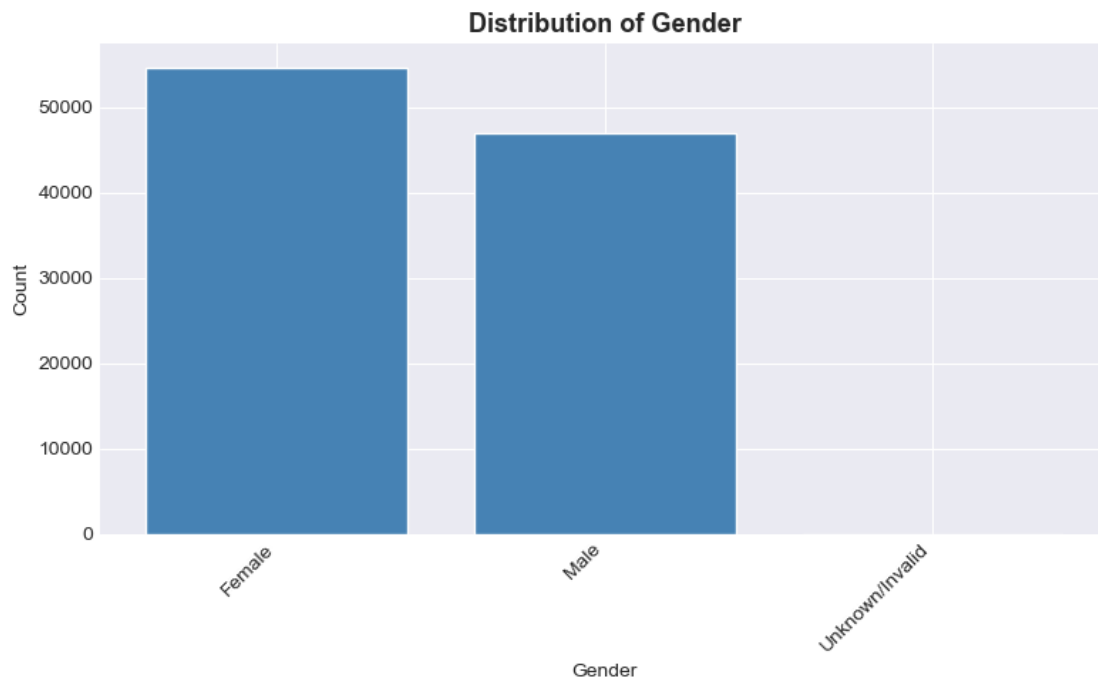
```
[6]: plt.bar(df['race'].unique(), df['race'].value_counts())
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.title('race')
plt.style.use('seaborn-v0_8-darkgrid')
plt.show()
```

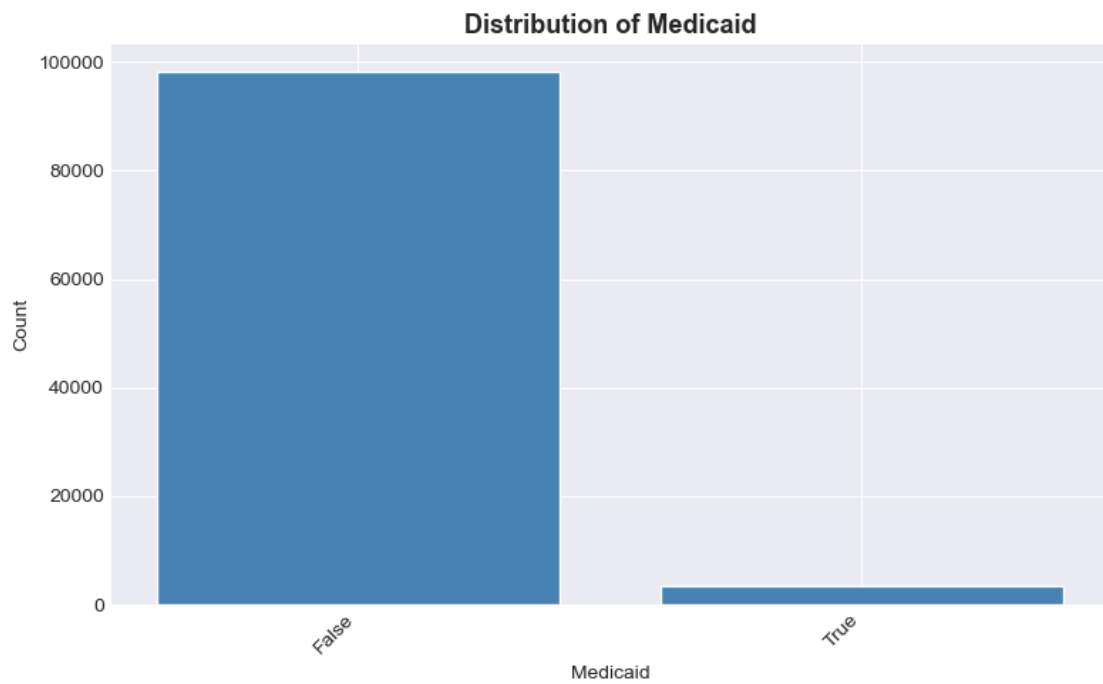
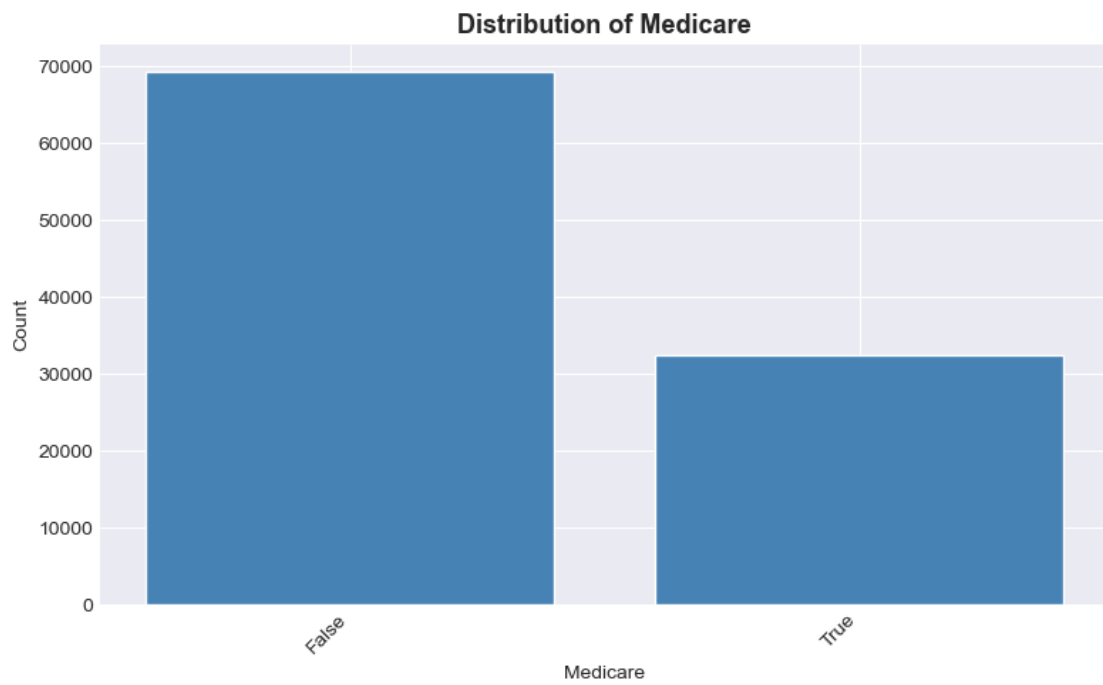


```
[7]: features = ['gender', 'age', 'medicare', 'medicaid']

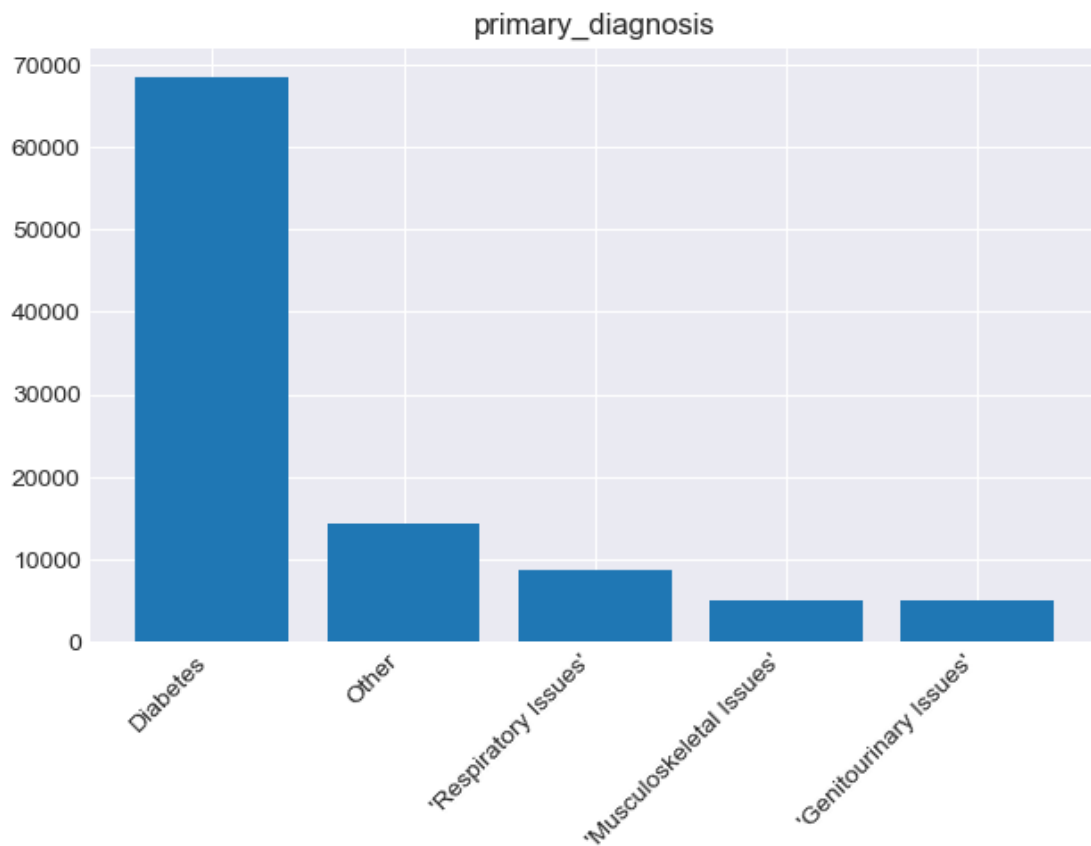
for feature in features:
    counts = df[feature].value_counts()
    x_labels = counts.index
    if feature in ['medicare', 'medicaid']:
        x_labels = counts.index.astype(str)
    plt.figure(figsize=(8, 5))
    plt.bar(x_labels, counts.values, color='steelblue', edgecolor='white')
    plt.title(f'Distribution of {feature.capitalize()}', fontsize=13,
fontweight='bold')
    plt.xlabel(feature.capitalize())
    plt.ylabel('Count')
```

```
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```





```
[8]: plt.bar(df['primary_diagnosis'].unique(), df['primary_diagnosis'].
        ↪value_counts())
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.title('primary_diagnosis')
plt.style.use('seaborn-v0_8-darkgrid')
plt.show()
```



1.0.5 Quantitative Variables (Boxplots & Histograms Side-by-Side)

Here we visualize the distribution of each quantitative variable using side-by-side boxplots and histograms.

```
[9]: import seaborn as sns
import matplotlib.pyplot as plt

quant_vars = ['time_in_hospital', 'num_lab_procedures', 'num_procedures',
        ↪'num_medications', 'number_diagnoses']

for var in quant_vars:
```

```

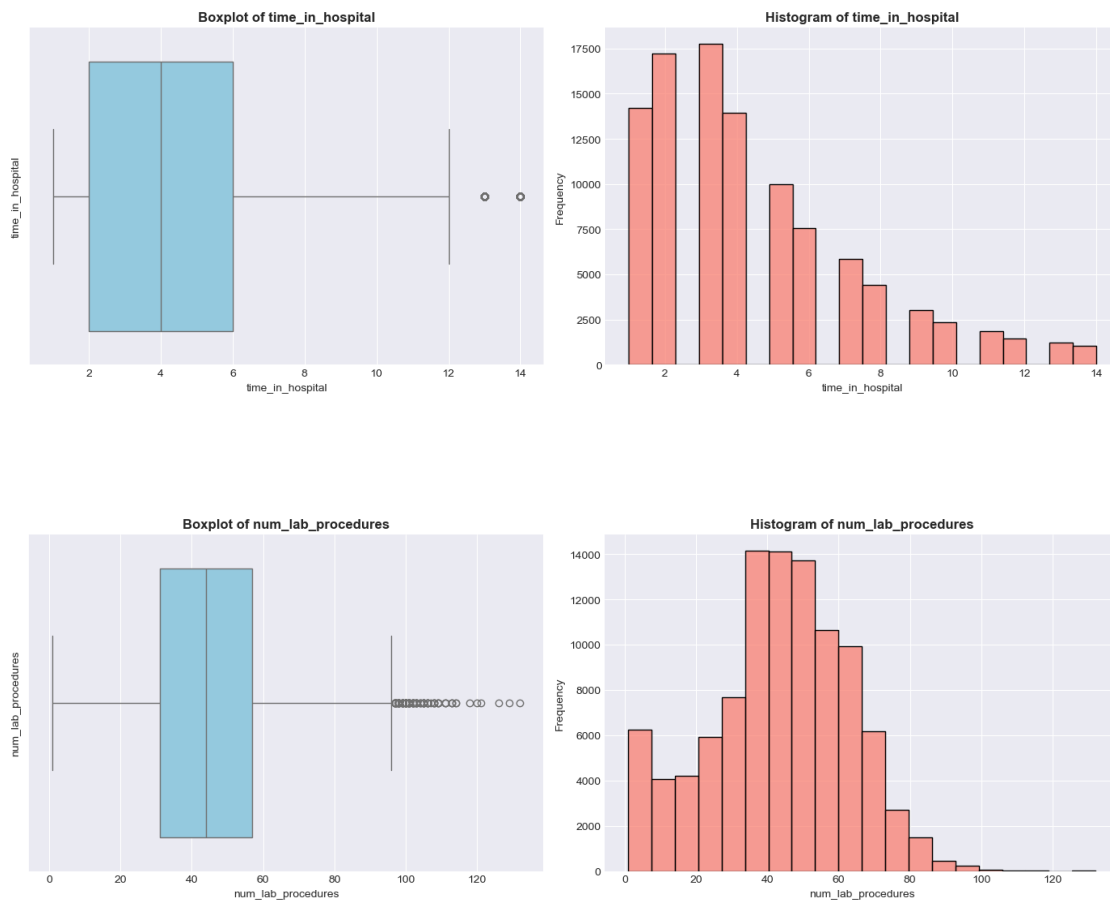
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

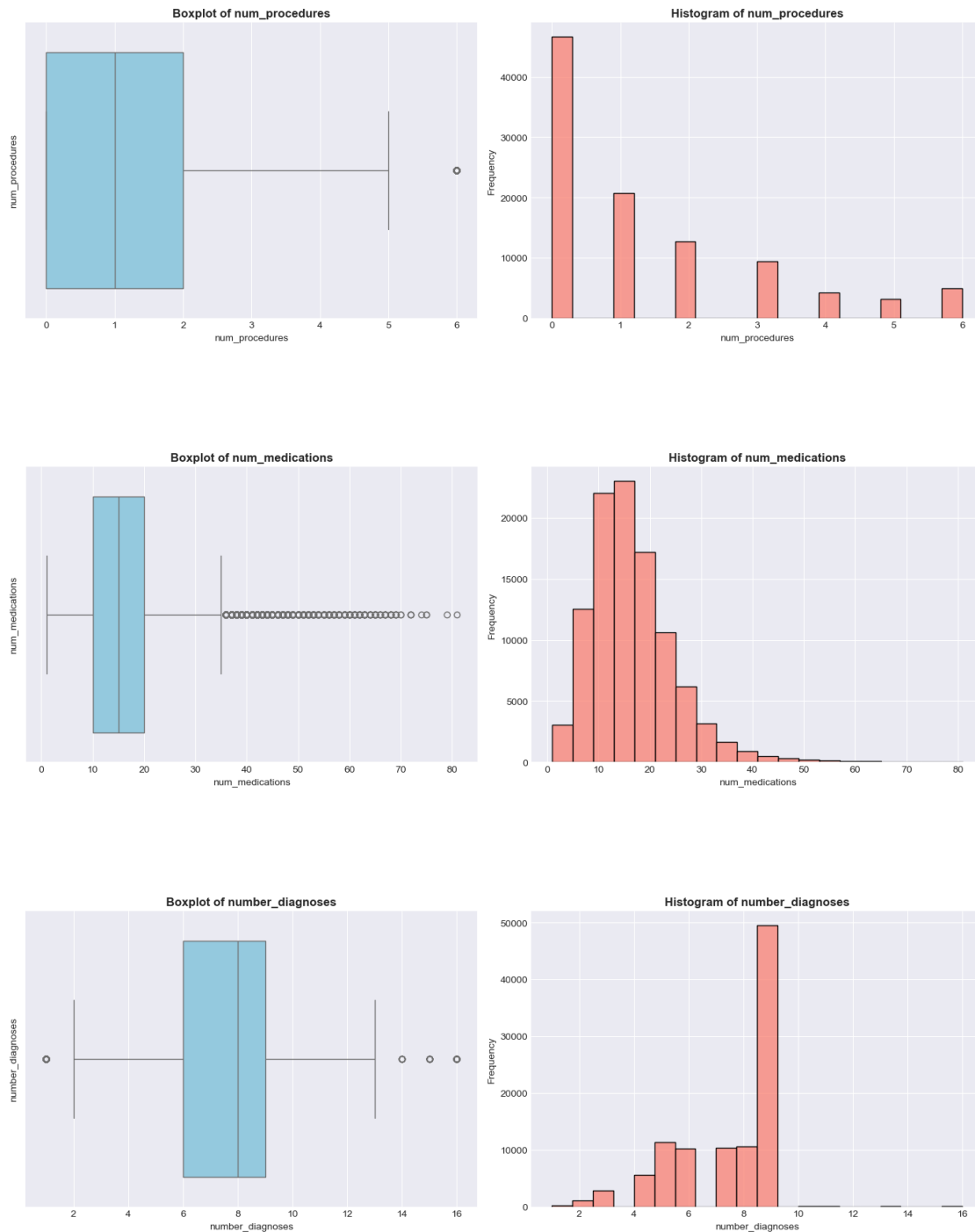
# Boxplot
sns.boxplot(x=df[var], ax=axes[0], color='skyblue')
axes[0].set_title(f'Boxplot of {var}', fontsize=12, fontweight='bold')
axes[0].set_ylabel(var)

# Histogram
sns.histplot(df[var], ax=axes[1], kde=False, color='salmon', bins=20)
axes[1].set_title(f'Histogram of {var}', fontsize=12, fontweight='bold')
axes[1].set_xlabel(var)
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

```



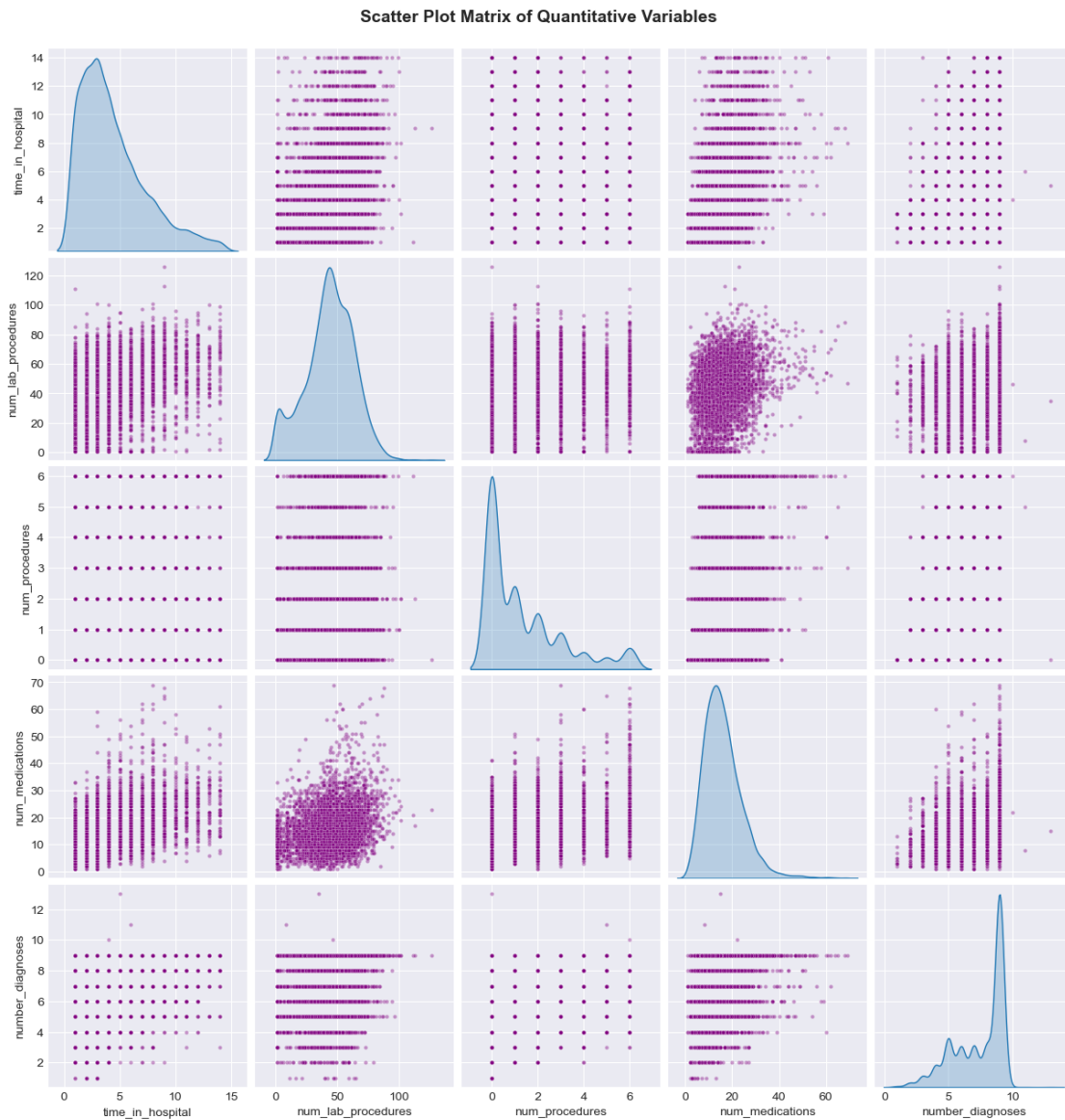


1.0.6 Scatter Plot Matrix of Quantitative Variable Pairs

We render a scatter plot matrix for all pairs of quantitative variables. We sample a subset of 5,000 observations to render the pairplot efficiently.

```
[10]: # Downsample to 5,000 points to render pairplot efficiently
sampled_df = df[quant_vars].sample(5000, random_state=42)

sns.pairplot(sampled_df, diag_kind='kde', plot_kws={'alpha': 0.4, 's': 10,
↪ 'color': 'purple'})
plt.suptitle('Scatter Plot Matrix of Quantitative Variables', y=1.02,
↪ fontsize=14, fontweight='bold')
plt.show()
```



1.0.7 Base Rate Disparities in Sensitive Attributes

To support fairness auditing, we compute and visualize the 30-day hospital readmission rate across different subgroups for each sensitive attribute.

1.0.8 Conditional Probabilities & Demographic Imbalances

The cross-tabulations computed below represent the exact definition of a **contingency table**. By looking at the raw counts of the target label (`readmit_30_days`) broken down by sensitive demographic features, we can understand the joint distribution $P(Y, A)$.

When we normalize these tables across the index, we calculate the **conditional probability** of readmission given the demographic group, $P(Y|A)$. Visualizing these readmission rates using bar plots allows us to quickly spot **demographic imbalances** and disparities in care outcomes before any machine learning takes place. If certain groups show significantly higher or lower readmission rates, it indicates that our training data is imbalanced and interventions like Inverse Probability Weighting (IPW) will be required to ensure algorithmic fairness.

```
[11]: from IPython.display import display, HTML

sensitive_attributes = ['race', 'gender', 'age', 'medicare', 'medicaid']
mycolors = ['deepskyblue', 'lightgrey', 'peachpuff', 'palegreen', 'lightpink']
t = 0

for attr in sensitive_attributes:
    # 1. Compute raw counts and normalized proportions
    raw_ct = pd.crosstab(df[attr], df['readmit_30_days'], margins=True,
    ↪margins_name='Total')
    norm_ct = pd.crosstab(df[attr], df['readmit_30_days'], normalize='index')

    # 2. Display them side-by-side using HTML
    html_content = f"""
    <div style='display: flex; justify-content: space-around; align-items:
    ↪center; margin-bottom: 20px;'>
        <div>
            <h3 style='text-align: center;'>{attr.capitalize()} (Raw Counts)</
    ↪h3>
            {raw_ct.to_html()}
        </div>
        <div>
            <h3 style='text-align: center;'>{attr.capitalize()} (Proportions)</
    ↪h3>
            {norm_ct.round(4).to_html()}
        </div>
    </div>
    """
    display(HTML(html_content))
```

```

# 3. Bar plots (1x2 Subplots)
ct_percentage = norm_ct * 100
counts_to_plot = raw_ct['Total'].drop('Total', errors='ignore')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

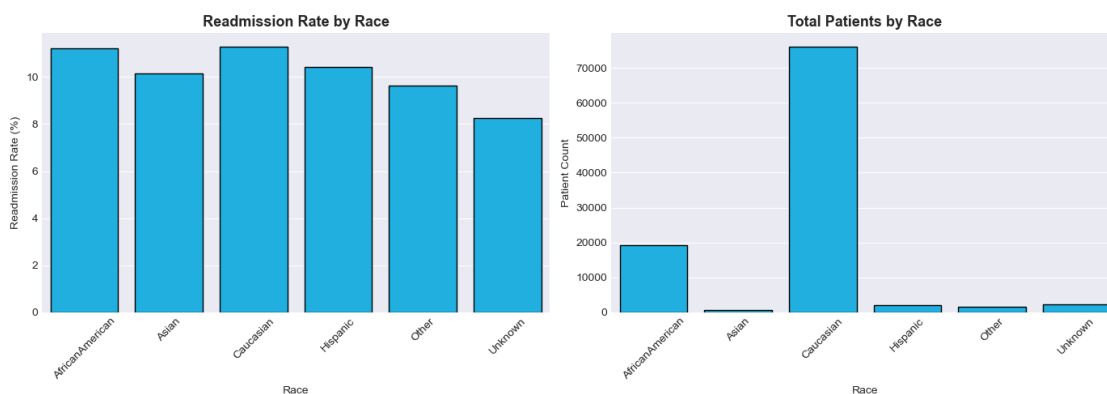
# Subplot 1: Readmission Rate
sns.barplot(x=ct_percentage.index, y=ct_percentage[1], color=mycolors[t],
            edgecolor='black', ax=axes[0])
axes[0].set_title(f'Readmission Rate by {attr.capitalize()}', fontsize=13,
                 fontweight='bold')
axes[0].set_xlabel(attr.capitalize())
axes[0].set_ylabel('Readmission Rate (%)')
axes[0].tick_params(axis='x', rotation=45)

# Subplot 2: Marginal Totals (Sample Size)
sns.barplot(x=counts_to_plot.index, y=counts_to_plot.values,
            color=mycolors[t], edgecolor='black', ax=axes[1])
axes[1].set_title(f'Total Patients by {attr.capitalize()}', fontsize=13,
                 fontweight='bold')
axes[1].set_xlabel(attr.capitalize())
axes[1].set_ylabel('Patient Count')
axes[1].tick_params(axis='x', rotation=45)

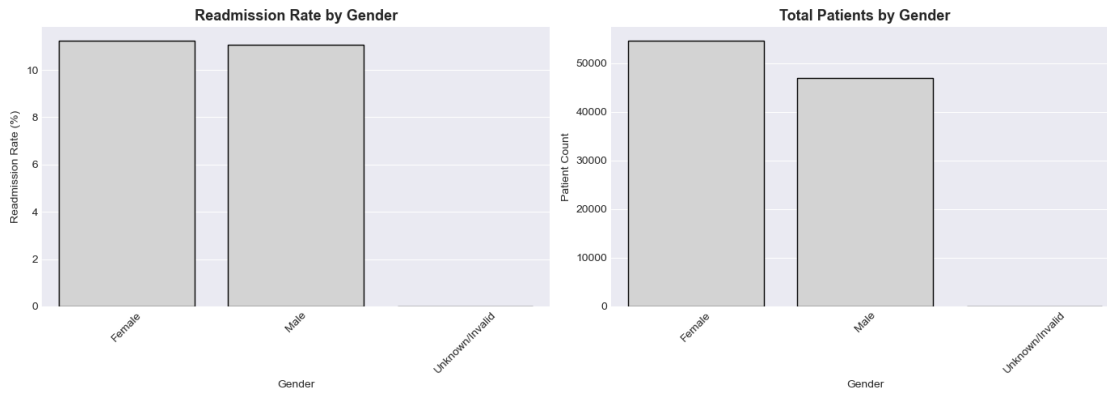
plt.tight_layout()
plt.show()
t += 1

```

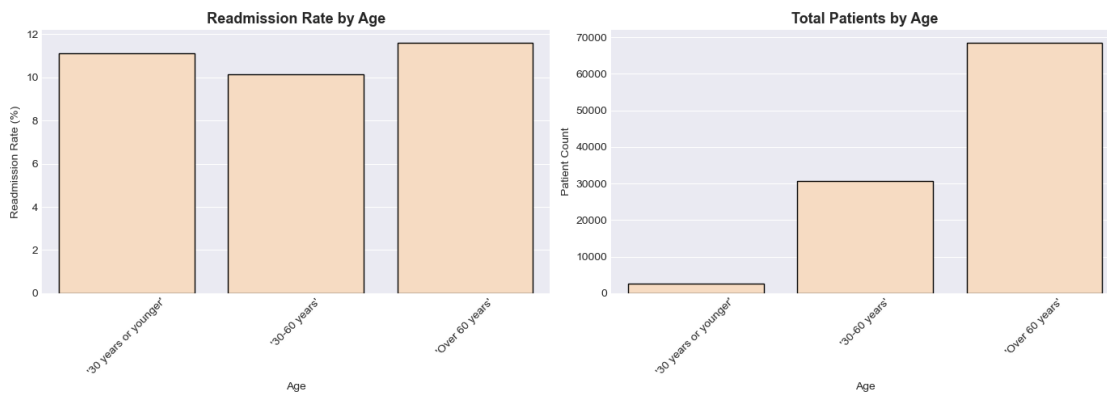
<IPython.core.display.HTML object>



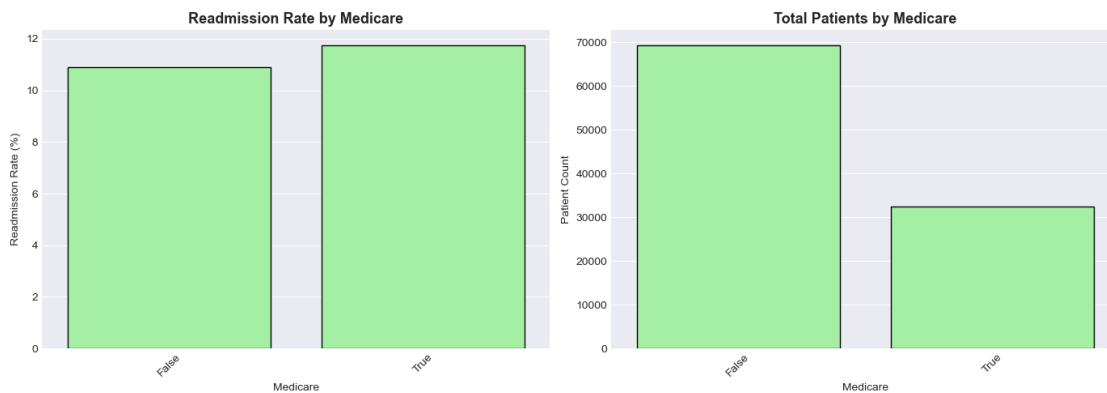
<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>



<IPython.core.display.HTML object>

